Unit 4

Network Programming and GUI using Python

Socket

- Socket programming is a way of connecting two nodes on a network to communicate with each other. One socket(node) listens on a particular port at an IP, while the other socket reaches out to the other to form a connection. The server forms the listener socket while the client reaches out to the server.
- They are the real backbones behind web browsing. In simpler terms, there is a server and a client.
  Socket programming is started by importing the socket library and making a simple socket.

- import socket
- s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
- Here we made a socket instance and passed it two parameters. The first parameter is **AF_INET** and the second one is **SOCK_STREAM**. AF_INET refers to the address-family ipv4. The SOCK_STREAM means connection-oriented TCP protocol.
  Now we can connect to a server using this socket.

Sockets have their own vocabulary:

Term	Description
domain	The family of protocols that is used as the transport mechanism. These values are constants such as AF_INET, PF_INET, PF_UNIX, PF_X25, and so on.
type	The type of communications between the two endpoints, typically SOCK_STREAM for connection-oriented protocols and SOCK_DGRAM for connectionless protocols.
protocol	Typically zero, this may be used to identify a variant of a protocol within a domain and type.
hostname	The identifier of a network interface : A string, which can be a host name, a dotted-quad address, or an IPV6 address in colon (and possibly dot) notation A string "<broadcast>", which specifies an INADDR_BROADCAST address. A zero-length string, which specifies INADDR_ANY, or An Integer, interpreted as a binary address in host byte order.
port	Each server listens for clients calling on one or more ports. A port may be a Fix num port number, a string containing a port number, or the name of a service

The socket Module

- To create a socket, you must use the socket.socket() function available in socket module, which has the general syntax:
- s = socket.socket (socket_family, socket_type, protocol=0)
  - socket_family: This is either AF_UNIX or AF_INET, as explained earlier.
  - socket_type: This is either SOCK_STREAM or SOCK_DGRAM.
  - protocol: This is usually left out, defaulting to 0.

Server Socket Methods

Method	Description
s.bind()	This method binds address (hostname, port number pair) to socket.
s.listen()	This method sets up and start TCP listener.
s.accept()	This passively accept TCP client connection, waiting until connection arrives (blocking).

Client Socket Methods

Method	Description
s.connect()	This method actively initiates TCP server connection.

General Socket Methods

Method	Description
s.recv()	This method receives TCP message
s.send()	This method transmits TCP message
s.recvfrom()	This method receives UDP message
s.sendto()	This method transmits UDP message
s.close()	This method closes socket
socket.gethostname()	Returns the hostname

IP using python

- You can also find the IP using python:

- import socket

- ip = socket.gethostbyname('www.google.com')
- print ip

What is URL ? How to read source code of webpage

- URL stands for Uniform resource locator.
- The requests module can help us build the URLS and manipulate the URLvalue dynamically.
- Any sub-directory of the URL can be fetched programmatically and then some part of it can be substituted with new values to build new URLs.

What is urllib?

- urllib is a Python module that can be used for opening URLs. It defines functions and classes to help in URL actions.
- With Python you can also access and retrieve data from the internet like XML, HTML, JSON, etc. You can also use Python to work with this data directly.
- In this tutorial we are going to see how we can retrieve data from the web.
- For example, here we used a guru99 video URL, and we are going to access this video URL using Python as well as print HTML file of this URL.

- import urllib.request
- url = "https://yashkotecha5.wordpress.com/"
- urllib.request.urlretrieve(url, "page.html")
- print("Web page downloaded using urllib!")

TCP/IP server and TCP/IP client

- Python is one of the most popular object-oriented scripting languages with a programmer-friendly syntax and a vast developer community.
- Here, we'll showcase how to write a TCP server and client in Python and Sockets can be configured to act as a server and listen for incoming messages , or connect to other applications as a client.
- After both ends of a TCP/IP socket are connected, communication is bi-directional.
- The sample contains the source code for a TCP server and client. For practice , you can extend it to build a small chat system or a local attendance tracking system.

Synchronous socket entities

- TCPServer class – It follows the (Internet) TCP protocol that allows continuous streams of data between the server and client.
- The socketserver module has more classes to handle sockets.
- Now let’s see the example of the Python TCP Server.

UDP server and UDP client.

- **UDP is USER DATAGRAM PROTOCOL**, this is a lightweight protocol which has basic error checking mechanism with no acknowledgement and no sequencing but very fast due to these reasons
- UDP or user datagram protocol is an alternative protocol to its more common counterpart TCP.
- UDP like TCP is a protocol for packet transfer from 1 host to another, but has some important differences.
- UDP is a connection-less and non-stream oriented protocol.
- It means a UDP server just catches incoming packets from any and many hosts without establishing a reliable pipe kind of connection.

Create UDP sockets

- A udp socket is created like this
- s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

User Datagram Client and Serve

- The user datagram protocol (UDP) works differently from TCP/IP.
- Where TCP is a stream oriented protocol, ensuring that all of the data is transmitted in the right order, UDP is a message oriented protocol.
- UDP does not require a long-lived connection, so setting up a UDP socket is a little simpler.
- On the other hand, UDP messages must fit within a single packet

sending E-mail using SMTP

- Simple Mail Transfer Protocol (SMTP) is a protocol, which handles sending e-mail and routing e-mail between mail servers.
- Python provides smtplib module, which defines an SMTP client session object that can be used to send mail to any Internet machine with an SMTP or ESMTP listener daemon.
- Here is a simple syntax to create one SMTP object, which can later be used to send an e-mail −
- **import smtplib**
- **smtpObj = smtplib.SMTP([host [, port [, local_hostname]]])**

Here is the detail of the parameters −

- host − This is the host running your SMTP server. You can specify IP address of the host or a domain name like tutorialspoint.com. This is optional argument.
- port − If you are providing host argument, then you need to specify a port, where SMTP server is listening. Usually this port would be 25.
- local_hostname − If your SMTP server is running on your local machine, then you can specify just localhost as of this option.
- An SMTP object has an instance method called sendmail

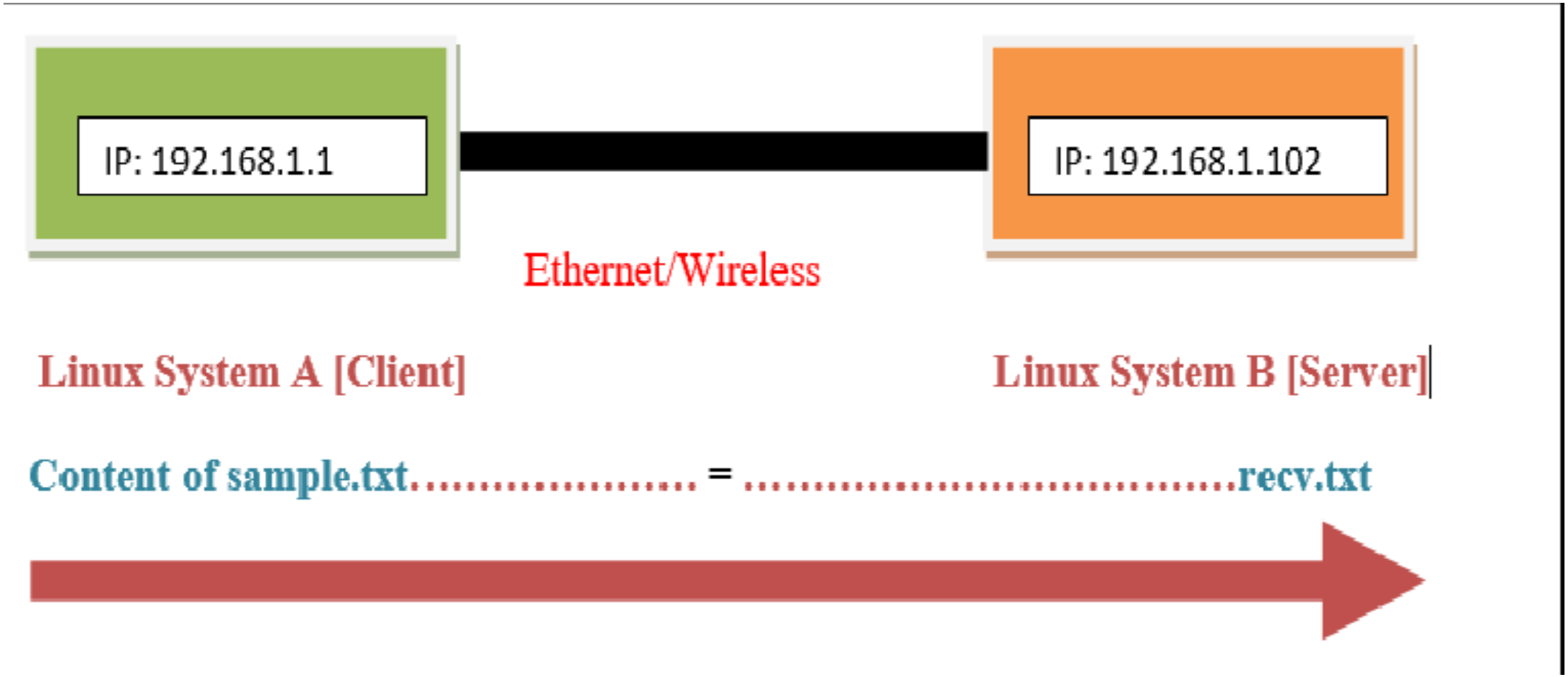
File server and File client

- A simple file transfer server written in Python 3, that allows the user to download files located on the server.
- The server can handle and serve multiple clients at the same time and send files in the same/child directories.

Usage

- Run the server.py file, entering the port you wish for the server to run on.
- Now, users can run the client.py file and connect to the server by entering the IP address and port displayed on your server.
- A user can enter the name of a file on the server (in the same folder as the server.py) and download that file - the file will appear in the same directory as the client.py file, named from_server+filename.
- file transfer is based on server client model to use socket programming in python3+.

Basic Set up Diagram:



File requirements:

- We need server.py and this file should be present at server system. In our case server.py should be at B_server system.
- Another two files client.py and sample.txt should be present at client system. In our case those two files should be present at A_client system
- File transfer is the process of copying or moving a file from a computer to another over a network or Internet connection.
- In this tutorial, we'll go step by step on how you can writeclient/server Python scripts that handles that.
- The basic idea is to create a server that listens on a particular port, this server will be responsible for receiving files (you can make the server sends files as well).
- On the other hand, the client will try to connect to the server and send a file of any type.
- Here is the code to send a file from a local server to a local client.

Two way communication between client and server

- Sockets are the endpoints of a bidirectional communications channel.
- Sockets may communicate within a process, between processes on the same machine, or between processes on different continents.
- So we will design both server and client model so that each can communicate with them. The steps can be considered like this.

1. Python socket server program executes at first and wait for any request
2. Python socket client program will initiate the conversation at first.
3. Then server program will response accordingly to client requests.
4. Client program will terminate if user enters “bye” message

PYTHON GUI PROGRAMMING :-

Event Driven Programming.

- Anything that happens in a user interface is an event.
- We say that an event is fired whenever the user does something – for example, clicks on a button or types a keyboard shortcut.
- Some events could also be triggered by occurrences which are not controlled by the user – for example, a background task might complete, or a network connection might be established or lost.
- Our application needs to monitor, or listen for, all the events that we find interesting, and respond to them in some way if they occur.

- To do this, we usually associate certain functions with particular events. We call a function which performs an action in response to an event an event handler – we bind handlers to events.
- Event-driven programming focuses on events. Eventually, the flow of program depends upon events.
- Until now, we were dealing with either sequential or parallel execution model but the model having the concept of event-driven programming is called asynchronous model.
- Event-driven programming depends upon an event loop that is always listening for the new incoming events.
- The working of event-driven programming is dependent upon events.
- Once an event loops, then events decide what to execute and in what order.

simple GUI

- GUI stands for Graphical User Interface.
- tkinter provides us with a variety of common GUI elements which we can use to build our interface – such as buttons, menus and various kinds of entry fields and display areas.
- We call these elements widgets.
- We are going to construct a tree of widgets for our GUI – each widget will have a parent widget, all the way up to the root window of our application.
- For example, a button or a text field needs to be inside some kind of containing window.
- The widget classes provide us with a lot of default functionality.

- They have methods for configuring the GUI's appearance – for example, arranging the elements according to some kind of layout – and for handling various kinds of user-driven events.
- Once we have constructed the backbone of our GUI, we will need to customise it by integrating it with our internal application class.
- Our first GUI will be a window with a label and two buttons:

- Python offers multiple options for developing GUI (Graphical User Interface).
- Out of all the GUI methods, tkinter is the most commonly used method.
- It is a standard Python interface to the Tk GUI toolkit shipped with Python.
- Python with tkinter is the fastest and easiest way to create the GUI applications. Creating a GUI using tkinter is an easy task.
- Importing tkinter is same as importing any other module in the Python code. Note that the name of the module in Python 2.x is ‘Tkinter’ and in Python 3.x it is ‘tkinter’.
- import tkinter

- There are two main methods used which the user needs to remember while creating the Python application with GUI

1. mainloop(): There is a method known by the name mainloop() is used when your application is ready to run.
    - mainloop() is an infinite loop used to run the application, wait for an event to occur and process the event as long as the window is not closed.
- m.mainloop()

2. m.quit

```
from tkinter import Tk, Label, Button

class MyFirstGUI:
    def __init__(self, master):
        self.master = master
        master.title("A simple GUI")

        self.label = Label(master, text="This is our first GUI!")
        self.label.pack()

        self.greet_button = Button(master, text="Greet", command=self.greet)
        self.greet_button.pack()

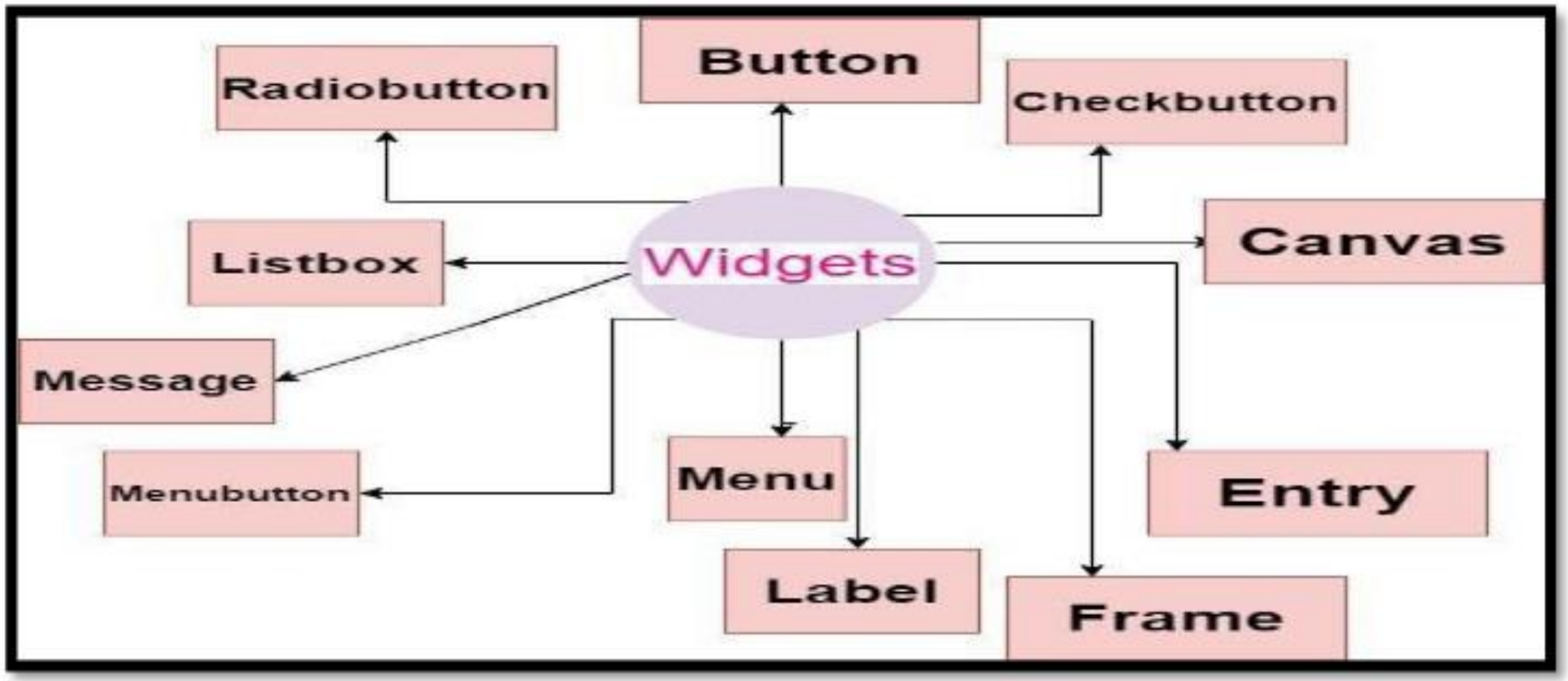
        self.close_button = Button(master, text="Close", command=master.quit)
        self.close_button.pack()

    def greet(self):
        print("Hello! Welcome to Tkinter GUI.")

root = Tk()
my_gui = MyFirstGUI(root)
root.mainloop()
```

Widgets in Python GUI

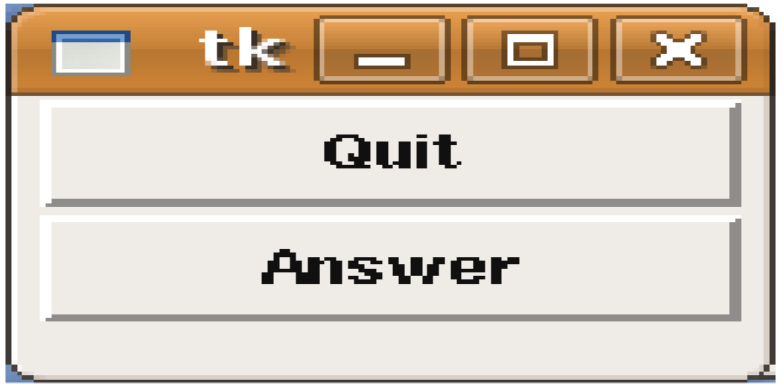
- Widgets are the bread and butter of the Python GUI framework Tkinter.
- They are the elements through which users interact with your program.
- Each widget in Tkinter is defined by a class. Here are some of the widgets available:
- There are various controls, such as buttons, labels, scrollbars, radio buttons, and text boxes used in a GUI application.
- These little components or controls of Graphical User Interface (GUI) are known as widgets in Tkinter.
- These are 19 widgets available in Python Tkinter module.


Radiobutton
Button
Checkbutton
Listbox
Widgets
Canvas
Message
Menubutton
Menu
Entry
Label
Frame

Widget	Class Description
Label	A widget used to display text on the screen
Button	A button that can contain text and can perform an action when Clicked
Entry	A text entry widget that allows only a single line of text
Text	A text entry widget that allows multiline text entry
Frame	A rectangular region used to group related widgets or provide Padding between widgets

Dialogues and Message boxes

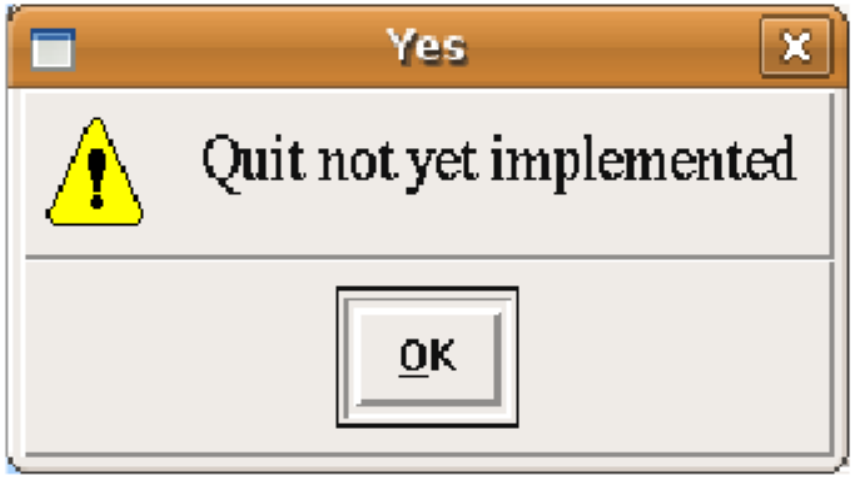
- Tkinter (and TK of course) provides a set of dialogues (dialogs in American English spelling), which can be used to display message boxes, showing warning or errors, or widgets to select files and colors .
- There are also simple dialogues, asking the user to enter string, integers or float numbers.
- Let's look at a typical GUI Session with Dialogues and Message boxes.
- There might be a button starting the dialogue, like the "quit" button in the following window:

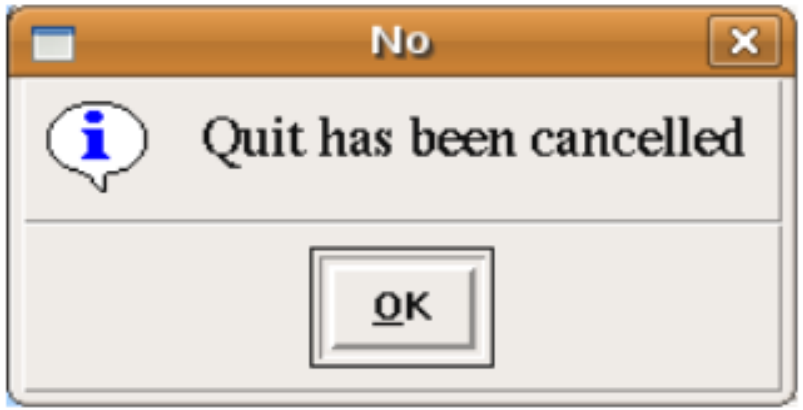


Pushing the "quit" button raises the Verify window:

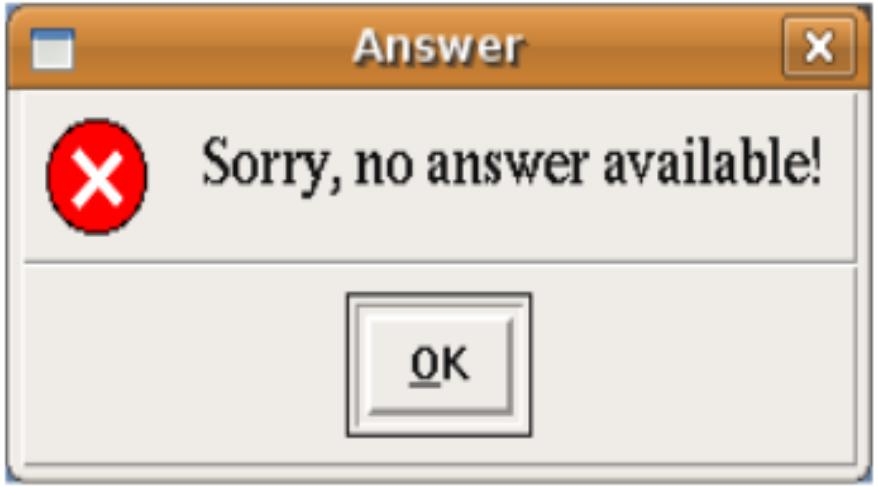


- Let's assume that we want to warn users that the "quit" functionality is not yet implemented. In this case we can use the warning message to inform the user, if he or she pushes the "yes" button





- Let's go back to our first Dialogue with the "quit" and "answer" buttons. If the "Answer" functionality is not implemented, it might be useful to use the following error message box:



```
import tkinter as tk
from tkinter import messagebox as mb

def answer():
    mb.showerror("Answer", "Sorry, no answer available")

def callback():
    if mb.askyesno('Verify', 'Really quit?'):
        mb.showwarning('Yes', 'Not yet implemented')
    else:
        mb.showinfo('No', 'Quit has been cancelled')

root = tk.Tk()
root.title("MessageBox Example")

tk.Button(root, text='Quit', command=callback).pack(fill=tk.X)
tk.Button(root, text='Answer', command=answer).pack(fill=tk.X)

root.mainloop()
```

Message Boxes

- The message dialogues are provided by the 'messagebox' submodule oftkinter.
- 'messagebox' consists of the following functions, which correspond to dialog windows:
- askokcancel(title=None, message=None, **options) Ask if operation should proceed; return true if the answer is ok
- askquestion(title=None, message=None, **options) Ask a question
- askretrycancel(title=None, message=None, **options) Ask if operation should be retried; return true if the answer is yes
- askyesno(title=None, message=None, **options) Ask a question; return true if the answer is yes
- askyesnocancel(title=None, message=None, **options) Ask a question; return true if the answer is yes, None if cancelled.

- showerror(title=None, message=None, **options) Show an error message
- showinfo(title=None, message=None, **options) Show an info message
- showwarning(title=None, message=None,**options) Show a warning message

Open File Dialogue

- There is hardly any serious application, which doesn't need a way to read from a file or write to a file.
- Furthermore, such an application might have to choose a directory.
- Tkinter provides the module tkFileDialog for these purposes.

```
import tkinter as tk
from tkinter import filedialog as fd

def callback():
    name = fd.askopenfilename()   # Open file dialog
    print(name)                   # Print the selected file path

root = tk.Tk()
root.title("File Dialog Example")

tk.Button(root, text='File Open', command=callback).pack(pady=10)

root.mainloop()
```

Choosing a Colour

- There are applications where the user should have the possibility to select a colour.
- Tkinter provides a pop-up menu to choose a colour. To this purpose we have to import the 'tkinter.colorchooser' module and have to use the method askColor:
- result = tkinter.colorchooser.askcolor (color, option=value, ...)
- If the user clicks the OK button on the pop-up window, respectively, the return value of askcolor() is a tuple with two elements, both a representation of the chosen colour, e.g. ((106, 150, 98), '#6a9662')
- The first element return[0] is a tuple (R, G, B) with the RGB representation in decimal values (from 0 to 255).
- The second element return[1] is a hexadecimal representation of thechosen colour.
- If the user clicks "Cancel" the method returns the tuple (None, None).
- The optional keyword parameters are:

color	The variable color is used to set the default colour to be displayed. If color is not set, the initial colour will be grey.
title	The text assigned to the variable title will appear in the pop-up window's title area. The default title is "Color".
parent	Make the pop-up window appear over window W. The default behaviour is that it appears over the root window.

```
import tkinter as tk
from tkinter.colorchooser import askcolor

def choose_color():
    color = askcolor(title="Pick a color")
    if color[1]:  # color[1] is the hex string (like "#ff0000")
        root.config(bg=color[1])  # Change background color
        print("Selected color:", color)

root = tk.Tk()
root.title("Color Chooser Example")
root.geometry("300x200")

tk.Button(root, text="Choose Color", command=choose_color,
fg="darkgreen").pack(pady=20)
tk.Button(root, text="Quit", command=root.quit, fg="red").pack(pady=10)

root.mainloop()
```

Widget Attributes

- In this part ,we talk about standard widget attributes, including cursors, reliefs, colours, and fonts.
- The standard widget attributes are keywords used in widget constructors.
- **Tkinter widget state**
- The state attribute defines the state of the widget. It can have the following values: NORMAL, ACTIVE, and DISABLED.
- We demonstrate the state attribute on the Label widget.
- label1 = Label(self, text=txt, state=NORMAL)

- **Tkinter Background colours**
- The background colours of widgets can be set with background attribute.
- It can be abbreviated to bg.
- Likewise, the foreground colours of widgets can be set with foreground attribute. It can be abbreviated to fg.
- In the code example, we create label with background colour.
- lbl1 = Label(frame, bg='SlateGray3', width=15, height=10)
- lbl1.pack(side=LEFT, padx=3)

- **Width and height**
- The width and height attributes set the width and height of the widget. In the example, we control the size of buttons with width and height.
- btn2 = Button(frame, text='Button', width=5 ,height =4)
- **Tkinter fonts**
- Tkinter has a tkinter.font module for working with fonts.
- It has some built- in fonts such as TkTooltipFont, TkDefaultFont, or TkTextFont.
- The font is set with the font attribute.
- Here we use a built-in font name.
- label3 = Label(self, text=txt, font=('Times', '18', 'italic'))

- **Tkinter cursors**
- The cursor is a small icon that shows where the mouse pointer is located. The cursor in Tkinter is set with the cursor attribute.
- In the example, we have four labels. Different cursors are used for each of the labels.
- lbl4 = Label(frame, bg='DarkSeaGreen4', width=15, height=10, cursor='pencil‘)

Layout Manager in tkinter

- When we design the GUI of our application, we decide what widgets we will use and how we will organize those widgets in the application.
- To organize our widgets, we use specialized non-visible objects called layout managers.
- There are two kinds of widgets: containers and their children.
- The containers group their children into suitable layouts.
- Tkinter has three built-in layout managers: the pack, grid, and place managers.
- The place geometry manager positions widgets using absolute positioning. The pack geometry manager organizes widgets in horizontal and vertical boxes. The grid geometry manager places widgets in a two dimensional grid

- **Absolute positioning**
- In most cases, programmers should use layout managers.
- There are a few situations where we could use absolute positioning.
- In absolute positioning, the programmer specifies the position and the size of each widget in pixels.
- The size and the position of a widget do not change if we resize a window. Applications look different on various platforms, and what looks OK on Linux, might not look OK on Mac OS.
- Changing fonts in our application might spoil the layout. If we translate our application into another language, we must redo our layout.

Button , Label ,Entryfield

- Button : To add a button in your application, this widget is used. The general syntax is: w=Button(master, option=value)
- master is the parameter used to represent the parent window. There are number of options which are used to change the format of the Buttons.
- Number of options can be passed as parameters separated by commas. Some of them are listed below.
- activebackground: to set the background color when button is under the cursor.
- activeforeground: to set the foreground color when button is under the cursor.
- bg: to set he normal background color. command: to call a function.
- font: to set the font on the button label. image: to set the image on the button.
- width: to set the width of the button. height: to set the height of the button.

- **CheckButton** : To select any number of options by displaying a number of options to a user as toggle buttons.
- The general syntax is: w = CheckButton(master, option=value)
- There are number of options which are used to change the format of this widget.
- Number of options can be passed as parameters separated by commas. Some of them are listed below.
- Title: To set the title of the widget.
- activebackground: to set the background color when widget is under the cursor.
- activeforeground: to set the foreground color when widget is under the cursor.
- bg: to set he normal backgrouSteganography Break.
- command: to call a function.
- font: to set the font on the button label.
- image: to set the image on the widget.

- **Entry** :It is used to input the single line text entry from the user.. For multi-line text input, Text widget is used.
- The general syntax is: w=Entry(master, option=value)
- master is the parameter used to represent the parent window.
- There are number of options which are used to change the format of the widget.
- Number of options can be passed as parameters separated by commas. Some of them are listed below.
- bd: to set the border width in pixels.
- bg: to set the normal background color.
- cursor: to set the cursor used.
- command: to call a function.
- highlightcolor: to set the color shown in the focus highlight.
- width: to set the width of the button.
- height: to set the height of the button.

- **Frame**: It acts as a container to hold the widgets. It is used for grouping and organizing the widgets.
- The general syntax is: w = Frame(master, option=value)
- master is the parameter used to represent the parent window.
- There are number of options which are used to change the format of the widget.
- Number of options can be passed as parameters separated by commas. Some of them are listed below.
- highlightcolor: To set the color of the focus highlight when widget has to be focused.
- bd: to set the border width in pixels.
- bg: to set the normal background color.
- cursor: to set the cursor used.
- width: to set the width of the widget.
- height: to set the height of the widget

- **Label**: It refers to the display box where you can put any text or image which can be updated any time as per the code.
- The general syntax is: w=Label(master, option=value)
- master is the parameter used to represent the parent window.
- There are number of options which are used to change the format of the widget.
- Number of options can be passed as parameters separated by commas. Some of them are listed below.
- bg: to set he normal background color.
- command: to call a function.
- font: to set the font on the button label.
- image: to set the image on the button.
- width: to set the width of the button.
- Height: to set the height of the button

- Listbox: It offers a list to the user from which the user can accept any number of options.
- The general syntax is: w = Listbox(master, option=value)
- master is the parameter used to represent the parent window.
- There are number of options which are used to change the format of the widget.
- Number of options can be passed as parameters separated by commas. Some of them are listed below.
- Highlightcolor : To set the color of the focus highlight when widget has to be focused.
- bg: to set he normal background color.
- bd: to set the border width in pixels.
- font: to set the font on the button label.
- image: to set the image on the widget.
- width: to set the width of the widget.
- height: to set the height of the widget.